

```

#include <iostream>
#include <string>
#include <ctime>

int main() {
    std::string name;
    int age;

    // Ask for the user's name
    std::cout << "Enter your name: ";
    std::getline(std::cin, name);

    // Ask for the user's age
    std::cout << "Enter your age: ";
    std::cin >> age;

    // Get the current year
    time_t now = time(0);
    tm* ltm = localtime(&now);
    int currentYear = 1900 + ltm->tm_year;

    // Calculate the year of birth
    int birthYear = currentYear - age;

    // Display the result
    std::cout << "Hello, " << name << "!
    You were born in " << birthYear << "." <<
    std::endl;

    return 0;
}

```

3. Compile and Run the Program:

- In your IDE, build the project to compile the code.
- Run the compiled program to execute it.
- Enter your name and age when prompted and verify the output.

Advanced C++ Features

C++ offers a range of advanced features that enable developers to write efficient and sophisticated code. Some of these features include templates, smart pointers, lambda expressions, and the Standard Template Library (STL).

Templates:

Templates allow you to write generic and reusable code. They are particularly useful for creating functions and classes that can operate with any data type.

```

#include <iostream>

// Function template
template <typename T>

```

```

T add(T a, T b) {
    return a + b;
}

int main() {
    int intSum = add<int>(3, 4);
    double doubleSum = add<double>(3.5,
    4.5);

    std::cout << "Integer sum: " << intSum
    << std::endl;
    std::cout << "Double sum: " << doubleSum
    << std::endl;

    return 0;
}

```

Smart Pointers:

Smart pointers, provided by the C++ Standard Library, manage the lifetime of dynamically allocated objects and help prevent memory leaks.

```

#include <iostream>
#include <memory>

class MyClass {
public:
    MyClass() {
        std::cout << "Constructor called"
        << std::endl;
    }

    ~MyClass() {
        std::cout << "Destructor called"
        << std::endl;
    }
};

int main() {
    {
        std::unique_ptr<MyClass> ptr =
        std::make_unique<MyClass>();
    } // Destructor is called automatically
    when ptr goes out of scope

    return 0;
}

```

Lambda Expressions:

Lambda expressions provide a concise way to define anonymous functions inline.

```

#include <iostream>
#include <vector>

```